Zcash Name Service

A Deterministic Name Registry over the Orchard Transaction Log

craftsoldier
github.com/craftsoldier julian@zcash.me

Protocol draft — August 25, 2026

Status: Draft
Category: Standards Track
Created: August 25, 2026
License: MIT
Discussion: github.com/craftsoldier/zns-whitepaper

Abstract

Zcash payments use encoded addresses, but Zcash consensus does not assign human-readable names to them. Existing on-chain naming systems cannot be ported to Zcash: transparent registries such as Namecoin publish the namespace in the clear, leaking the address graph Zcash exists to hide; contract-based registries such as ENS require general-purpose on-chain state that Zcash consensus does not provide. The Zcash Name Service (ZNS) maps each Zcash name to a Unified Address over the ordinary Zcash transaction log: each binding is recorded in the memo of an Orchard action, decryptable with the Name Note account's published full viewing key, and anchored by an Orchard note commitment derived from the same tuple. Zcash consensus supplies canonical order and finality; a deterministic reducer derives the registry; the bound Unified Address inherits Orchard shielding. Authorization is delivered by an attested Mint; a future off-chain zero-knowledge name circuit would replace that assumption with a proof.

Status. This document is a work in progress. Sections explicitly marked Open or Provisional are not interoperability requirements. Sections

marked FIXED state normative requirements backed by the cited assumptions. An implementation cannot infer a protocol rule from an unresolved item. The key words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** are to be interpreted as described by BCP 14 only when they appear in uppercase [5].

1 Motivation

Zcash addresses are long encoded strings. A naming system maps these strings to human-readable labels, but this requires enforcing rules about which names are claimed and who controls them. Zcash consensus tracks unspent coins. Lacking a virtual machine to run logic and a global state tree to store variables, it cannot evaluate a naming policy. To bypass this limitation, ZNS separates the execution of the naming policy from the permanent public record. A single party, the Mint, executes the policy off-chain. When a request passes, the Mint writes the resulting name binding into the 512-byte memo field of a shielded Zcash transaction. Zcash consensus seals the transaction into a block, locking the binding into a permanent, chronological order. A Resolver looking up a name downloads the Zcash block headers, uses a published viewing key to decrypt the Mint's memos, and reads the bindings in the exact order consensus finalized them. The Mint issues names, but the accumulated work of the Zcash blockchain prevents it from altering or erasing previously recorded bindings.

2 Architecture

To guarantee the globally unique mapping of human-readable names to Zcash addresses, a naming system requires an agreed-upon history of requests to register, modify, or release names, a strict set of rules, and a resulting registry of name bindings. ZNS treats this registry as derived state by using Zcash exclusively for the history of these requests, executing rules off-chain, and binding their accepted results on-chain.

2.1 System Components

The protocol relies on three distinct components:

The Mint. The authoritative off-chain program that evaluates the naming policy. It runs inside a Trusted Execution Environment (TEE), with its authority bound to approved protocol code through remote attestation. It uses its own Zcash node to read chain state independently.

The Name Note. An Ironwood output used to record a name transition on-chain.

The Resolver. A permissionless client that reads the canonical sequence of Zcash transactions, verifies Name Notes and deterministically derives registry state.

2.2 Independence of Facts

The security of the derived state relies on six distinct properties. The Resolver verifies publicly reconstructible properties directly and relies on Mint attestation for policy checks that are not publicly reconstructible.

Zcash validity: The transaction satisfies Zcash consensus.

Mint authorization: The recognized Mint spending key authorized the transaction.

TEE attestation: The recognized Mint identity executed the approved enclave code.

Name Note verification: The transition tuple encoded in a Name Note matches its Ironwood note commitment.

User authorization: The attested Mint enforces the authorization procedure defined in Section 5.

State derivation: Resolvers reading the canonical Zcash chain under identical protocol parameters derive the same registry state.

3 Name Notes

A Name Note is an Ironwood output whose memo records a name transition by encoding the type of name change, the name, the associated Unified Address when applicable, the expiration value, and a predecessor reference.

3.1 Memo encoding

The Name Note memo is a 512-byte field. A parser removes its maximal trailing sequence of zero bytes. The remaining bytes encode one transition in one of the following formats:

```
ZNS:claim:<name>:<ua>:<expires_at>:<prev_rcm>
ZNS:update:<name>:<ua>:<expires_at>:<prev_rcm>
ZNS:release:<name>:<ua>:none:<prev_rcm>
```

The colon byte (`0x3a`) separates fields. The `<expires_at>` field is either the canonical ASCII decimal encoding of a Unix timestamp in whole seconds or the exact ASCII bytes `none`. The value `none` indicates that the registration has no fixed expiration. A `release` MUST encode `none` as its `expires_at` value and the Unified Address of the binding being released as its `ua` value.

The `prev_rcm` field is the 64-character lowercase hexadecimal encoding of the predecessor Name Note's `rcm` value. The first claim uses 64 zeroes.

3.2 Transition representation

To derive the ZNS-specific note-commitment inputs, rcm_σ and ψ_σ, interpret the encoded transition as the tuple

$$\sigma = (\alpha, n, u, e, p)$$

where:

- α: The exact ASCII bytes `claim`, `update`, or `release`.
- n: The raw bytes of the name field.
- u: The raw bytes of the `ua` field.
- e: The raw bytes of the `expires_at` field.
- p: The 32 raw bytes decoded from `prev_rcm`.

3.3 Commitment inputs

ZNS derives both commitment inputs from the same transition representation σ using the same hash construction. Within the hash construction, the

only difference is the ASCII derivation tag, `rcm` or `psi`. The resulting digests are then reduced into different Pallas fields.

To prevent ambiguous concatenation, define $\mathsf{LP}(x)$ as the four-byte unsigned little-endian byte length of x, followed by x itself. Define the protocol domain tag T as the 12 ASCII bytes `ZcashName/v1`. Quotation marks are not part of T.

For derivation tag t, where t is the exact ASCII byte string `rcm` or `psi`, define:

$$H_t(\sigma) = \mathsf{BLAKE2b{-}512}(\mathsf{LP}(T) \parallel \mathsf{LP}(t) \parallel \mathsf{LP}(\alpha) \parallel \mathsf{LP}(n) \parallel \mathsf{LP}(u) \parallel \mathsf{LP}(e) \parallel p).$$

Because e is included in σ, both ZNS-specific commitment inputs cryptographically bind the `expires_at` value recorded in the Name Note.

The final p is exactly 32 raw bytes and has no length prefix. Each H_t invocation uses unkeyed BLAKE2b-512 with a 64-byte output and no additional personalization.

The two commitment inputs are then:

$$\mathsf{rcm}_\sigma = \mathsf{ToScalar}(H_{\texttt{rcm}}(\sigma)), \quad \psi_\sigma = \mathsf{ToBase}(H_{\texttt{psi}}(\sigma)).$$

The value rcm_σ is the Pallas scalar-field element used as the Ironwood note-commitment randomness.

The value ψ_σ is the Pallas base-field element supplied to the Ironwood note commitment construction.

$\mathsf{ToScalar}$, ToBase, and the field types are those defined for Ironwood by the Zcash Protocol Specification [1, 3]. Neither rcm_σ nor ψ_σ is a raw 64-byte BLAKE2b digest; each is the field element obtained by reducing the digest produced with its respective derivation tag. The canonical encoding of either field element is its 32-byte little-endian field representation. ZNS derives both field elements from σ, rather than from the *rseed* derivation used by ordinary ZIP-212 receiving [2, 1].

Changing the domain tag, a derivation tag, field order, length width, byte order, hash parameters, or reduction rule changes the protocol version and requires new test vectors.

3.4 Commitment verification

ZNS changes the derivation of ψ and rcm but does not otherwise change the standard note commitment construction. A verifier derives ψ_σ and rcm_σ from the encoded transition, combines them with the candidate's remaining note components using the standard construction [1, 3], and reconstructs the note commitment.

The Name Note passes commitment verification only if the reconstructed value equals the `cmx` recorded in the on-chain action.

This equality establishes only that the transition encoded in the memo is cryptographically bound to that output. It does not establish Mint authorization, user authorization, or the current registry state.

3.5 Commitment test vector

For a `claim` transition with `name=alice`, `expires_at=none`, and a 32-byte zero predecessor. Key material is derived from the all-zero 32-byte seed under unified account $\mathtt{m}/32'/133'/1'$, diversifier index 0, external scope, mainnet:

Value	Canonical little-endian bytes rendered as hexadecimal
`ua`	`u1897y9pzw3zk6n9twtzu2z5kpkzw3hms2c54fpyv8lnr79m73tazljkk3veaxrtwncp66lf45p3f2`
`g_d`	`de4338f2ab9fd8300a3a1c20dd690ce27026c6001c295d7c641a067ce809b11e`
`pk_d`	`6df609f5710f3b5deecd4ee4b8f0173b44af6cf8918ac00269526031ba628996`
ψ_σ	`9f8a61b860c737d4564f12c635d654b843bc7115d9dc6cf6f09e409c81b8d13e`
rcm_σ	`daa928be21d0ec13b5dbb0244699dbfeba546c71591d24d7824db78e4670c504`

Set the value to zero and ρ to 32 bytes of `0x33`. The resulting `cmx` is `cc320736a0c1df1e4ffcee2b64aa73a9e6d06bb218e155a6fef422e1ecb1f70c`. An implementation that produces another value is non-conforming.

The complete memo encoding of this transition is:

```
ZNS:claim:alice:u1897y9pzw3zk6n9twtzu2z5kpkzw3hms2c54fpyv8lnr79m73tazljkk3veaxrtwncp6
```

4 The Mint

The Mint is the protocol's single registrar. A Name Note establishes that a transition is bound to an Ironwood output, but whether the transition is allowed depends on the current registry state and active naming policy. The Mint reads ordered requests on Zcash, derives that state, applies the policy, and for each accepted request spends a Mint-controlled note to create the successor Name Note.

4.1 Authority bound to policy

The Mint spending key provides the protocol's Mint authorization. The key can create a transition that appears to have been approved by the Mint without applying the naming policy. The key must therefore be usable only by the program that evaluates the policy.

ZNS runs the Mint inside a Trusted Execution Environment (TEE) and uses remote attestation to bind the Mint key to an approved enclave measurement. This shifts trust in policy execution and key custody from the Mint operator to the TEE attestation system and its hardware root of trust.

4.2 Attestation constraints

The Resolver recognizes Mint authorization only if the attestation establishes all of the following:

1. The enclave code measurement is approved by the current ZNS deployment.
2. The enclave controls the recognized Mint spending key.
3. The enclave runs the applicable ZNS protocol version.
4. The enclave targets the applicable Zcash network.
5. The enclave implements the required protected persistent-state mechanism for replay prevention.
6. The enclave obtains canonical-chain Median Time Past from the approved Zcash node interface and uses it as the authoritative time source for protocol-defined time checks.

The Resolver rejects the Mint authorization if any condition fails or if the TEE is not running in an approved production security configuration.

Approved enclave measurements, attestation requirements, and the approved source of canonical-chain Median Time Past are deployment-specific. The exact attestation format and verification procedure depend on the selected TEE platform and remain OPEN until the deployment is fixed.

4.3 Claim eligibility

Some names may be designated as protected for a limited protection period to reduce impersonation, misleading claims of affiliation, and opportunistic registration of names associated with existing identities or projects.

A protected name is temporarily excluded from the standard public claim process. During the protection period, a claim must include a valid access code, which the Mint verifies according to the protected-name policy before accepting the claim.

Access-code verification is performed by the Mint under the attested protected-name policy and is not independently reconstructed by the Resolver.

When the protection period ends, an unclaimed protected name becomes eligible for the standard public claim process. Once a protected name is validly claimed, it is governed by the same transition and authorization rules as any other name.

4.4 Request evaluation

The Mint evaluates each request against the registry state it derives from the canonical Zcash chain. It must use its own Zcash node and its own replay of the namespace history rather than rely on a third-party state oracle. The Mint obtains canonical-chain Median Time Past from that node for protocol-defined time checks.

The Mint evaluates each request against the following requirements:

- **Namespace state.** The requested change must be permitted by the current state of the name.
- **Payment.** The request must satisfy the price and payment requirements defined by the active pricing policy.
- **Request validity.** The name, Unified Address when applicable, requested registration term or extension when applicable, and resulting

encoded transition must satisfy the protocol's validity rules.

- **Lifecycle policy.** The request must satisfy the active registration, expiration, renewal, and liveness rules defined in Section 4.5.
- **User authorization.** An `update` or controller-requested `release` must complete the authorization procedure defined in Section 5. A lifecycle `release` created by the Mint to enforce expiration or liveness does not require user authorization.

If all applicable requirements are satisfied, the Mint accepts the request and creates the resulting Name Note.

4.5 Name lifecycle

ZNS uses canonical-chain Median Time Past (MTP) rather than block counts or local system time for protocol-defined lifecycle periods. MTP is derived from Zcash block timestamps and has one-second granularity. It is chain-derived time rather than exact wall-clock time.

4.5.1 Initial claims

A `claim` request specifies the name, the Unified Address, and either a requested registration term or no fixed expiration. The user does not supply an absolute `expires_at` value.

For a fixed-term registration, the Mint computes `expires_at` by adding the requested registration term to the canonical-chain MTP of the block containing the accepted claim request.

The Mint determines `expires_at` from that block MTP and the requested registration term, subject to the active registration policy, and records the resulting value in the Name Note.

For a registration without a fixed expiration, `expires_at` is the exact ASCII value `none`.

Only the resulting `expires_at` value is included in the Name Note. The claim request's block MTP and requested registration term are policy inputs and are not repeated as transition fields.

4.5.2 Expiration

For a fixed-term registration, the expiration condition is reached when:

$$\text{canonical\_chain\_mtp} \geq \text{expires\_at}.$$

Reaching `expires_at` does not directly modify derived registry state. When the expiration condition is reached, the Mint MUST create a `release` Name Note.

The registration remains active in derived registry state until a `release` Name Note is accepted on the canonical Zcash chain, subject to the same-block precedence rule defined in Section **??**.

A registration whose `expires_at` value is `none` has no fixed expiration but remains subject to the liveness requirement.

4.5.3 Updates and renewals

An ordinary `update` does not require the user to provide `expires_at`. The Mint reads the current accepted registration state and carries the current `expires_at` into the successor Name Note.

An ordinary `update` MUST NOT extend, shorten, restart, or remove the current registration period.

An `update` MAY request a registration extension. The user supplies the requested additional term, not an absolute expiration timestamp.

For a finite registration:

$$\text{new\_expires\_at} = \text{current\_expires\_at} + \text{requested\_term}.$$

If no extension is requested:

$$\text{new\_expires\_at} = \text{current\_expires\_at}.$$

If the current `expires_at` is `none`, an ordinary term extension does not change it.

The extension is added to the existing `expires_at`, rather than to the current time.

Once the expiration condition has been reached, the Mint MUST NOT accept a new renewal request and MUST create the required `release` Name Note.

An update or renewal affects the registration only if its Name Note is accepted in a block preceding any `release` Name Note that ends the registration.

If an `update` and `release` for the same current registration are accepted in the same block and both reference the same predecessor, the `release` takes precedence regardless of the update's purpose or resulting `expires_at`.

If the `release` is accepted in an earlier block, the registration has ended and a later `update` does not affect registry state.

An `update` MAY retain the currently bound Unified Address. Such an update may be used for renewal, liveness confirmation, or both.

4.5.4 Liveness

The Mint is responsible for tracking and enforcing liveness.

The current ZNS deployment MUST fix the liveness interval L in seconds.

For a registration whose most recent liveness time is τ, the liveness deadline is:

$$\text{liveness\_deadline} = \tau + L.$$

The initial liveness interval begins when the `claim` Name Note is accepted on the canonical Zcash chain. A successfully authorized `update` resets the liveness interval when the resulting Name Note is accepted on the canonical chain, including an update that retains the currently bound Unified Address.

The liveness time of a `claim` or `update` is derived from the canonical-chain MTP associated with the block containing that accepted Name Note.

The Mint records the most recent successful liveness event in protected persistent state and derives the next liveness deadline using canonical-chain MTP. If a chain reorganization removes or changes a `claim` or `update` used

to establish the current liveness time, the Mint MUST reconcile its protected lifecycle state with the replacement canonical chain before enforcing the resulting liveness deadline.

Requesting an OTP does not satisfy liveness. A failed, expired, or incomplete OTP exchange does not satisfy liveness. A liveness interval resets only when the resulting `update` Name Note is accepted on the canonical Zcash chain.

If the required liveness interval passes without a successful liveness event, the Mint MUST create a `release` Name Note.

For a fixed-term registration, the Mint MUST create a release when either the committed `expires_at` is reached or the liveness requirement fails, whichever occurs first.

An `update` affects the registration only if it is accepted in a block preceding the required `release`. If the update and release are accepted in the same block and reference the same current predecessor, the release takes precedence.

Liveness enforcement is performed by the attested Mint and is not part of the Resolver's required state-derivation procedure.

4.5.5 Release conditions

The Mint MUST create a `release` Name Note when any of the following occurs:

1. the controller successfully completes the required authorization procedure for an explicit `release`;
2. canonical-chain MTP reaches the committed `expires_at`; or
3. the current liveness deadline is reached without an accepted `update` Name Note having reset the liveness interval.

The first condition is a controller-requested release and requires the OTP authorization procedure defined in Section 5. The second and third conditions are lifecycle releases created by the Mint and do not require user authorization.

Reaching an expiration or liveness deadline does not itself modify registry state. Registry state changes only when the resulting `release` Name Note is accepted on the canonical Zcash chain.

An `update` affects the current registration only if it is accepted in a block preceding the `release`. If an `update` and `release` referencing the same current predecessor are accepted in the same block, the `release` takes precedence.

After a `release`, a subsequent `claim` begins a new registration and uses the zero predecessor defined in Section 3.

4.6 On-chain completeness

A Resolver must be able to reconstruct registry state from the on-chain Name Note history without relying on the Mint as a data source. Every accepted transition must therefore be fully represented in its Name Note. The Mint must not omit a transition field or replace it with an off-chain pointer.

For a `claim` or `update`, the resulting `expires_at` is part of the transition and MUST be present in the Name Note. The requested registration term, requested extension, and MTP values used by the Mint to apply lifecycle policy are policy inputs and need not be repeated in the Name Note.

For a `release`, the Name Note records the Unified Address of the binding being released.

The protocol's validity limits on names, Unified Addresses, and other encoded fields MUST ensure that every valid ZNS transition fits within a single 512-byte Name Note memo.

5 User Authorization

Zcash shielded addresses do not provide a standard message-signing mechanism for authorizing changes to an existing name. ZNS therefore uses an in-band one-time passcode (OTP) delivered through Zcash's native shielded encryption.

For an `update` or controller-requested `release`, the Mint sends a fresh random challenge to the Unified Address currently bound to the name. The OTP is bound to the requested transition, expires after a fixed time interval measured using canonical-chain MTP, and is consumed on successful use. The controller returns the OTP to the Mint by sending it in a shielded memo to the Mint's designated admin/treasury wallet. Receipt of the correct OTP

within that window demonstrates both the ability to decrypt shielded messages sent to the selected receiver and temporal liveness.

A lifecycle `release` created by the Mint to enforce expiration or liveness does not require an OTP.

A `claim` has no existing controller and therefore requires no OTP.

5.1 Authorization target

For an `update` or controller-requested `release`, the Mint sends the OTP to the Ironwood receiver contained in the Unified Address currently bound to the name. The Mint MUST reject authorization if that Unified Address does not contain an Ironwood receiver. Successful authorization establishes access to that receiver only, not to any other receiver contained in the Unified Address.

For an update, authorization does not verify control of the proposed new Unified Address. A valid but unintended or substituted Unified Address can therefore become the new binding.

An update MAY specify the currently bound Unified Address as the target address when performed for renewal, liveness confirmation, or both.

5.2 OTP relay memo

The Mint delivers the OTP in an Ironwood shielded memo. The non-padding bytes are:

```
ZNS:otp:<name>:<verb>:<ua>:<otp>
```

The `<verb>` field is `update` or `release`. The `<ua>` field is the target Unified Address from the request. The `<otp>` field is exactly six ASCII decimal digits, including leading zeroes.

The memo is exactly 512 bytes and zero-padded. The `otp` message type distinguishes it from a name request, and a request parser MUST reject it as such.

OTP relay memos are sent from a separate Mint account whose viewing key is not public.

5.3 OTP validity

The Mint generates each OTP uniformly from `000000` through `999999` using a cryptographically secure random source inside the attested environment [7].

For an `update`, the Mint computes the resulting `expires_at` before issuing the OTP and binds the OTP to the exact `(name, action, ua, expires_at)` tuple in its protected state.

For an ordinary update, `expires_at` is the current accepted expiration value carried forward unchanged. For a renewal, `expires_at` is the value obtained by adding the requested additional term to the current accepted expiration.

The user is not required to supply the resulting absolute `expires_at` as a request field. The Mint MUST accept the OTP only for the same pending update and MUST reject it if the resulting `(name, action, ua, expires_at)` would differ from the tuple to which the OTP was issued.

For a controller-requested `release`, the OTP remains bound to the applicable `(name, action, ua)` values. Lifecycle releases created by the Mint to enforce expiration or liveness do not use OTP authorization.

The current ZNS deployment MUST fix the OTP validity duration D_{OTP} in seconds and the maximum number of verification attempts.

If an OTP is issued when canonical-chain MTP is τ, define:

$$\mathrm{otp\_expires\_at} = \tau + D_{\mathrm{OTP}}.$$

The Mint MUST reject the OTP when canonical-chain MTP is greater than or equal to `otp_expires_at`.

The Mint MUST also reject an OTP presented for a different bound transition, an OTP that exceeds the permitted number of attempts, or an OTP that has already been successfully used. A successfully used OTP is consumed and MUST NOT be accepted again for the same request.

OTP expiration is independent of registration expiration and the liveness deadline.

Under the TEE assumptions, attestation establishes that the Mint executed the approved authorization procedure.

5.4 Public verification boundary

The OTP exchange does not produce public evidence of user authorization. A Resolver can verify the resulting Name Note, including the `expires_at` value committed by an `update`, as well as Mint authorization and attestation, but it cannot reconstruct the OTP exchange or independently verify that the OTP was bound to that transition from the on-chain Name Note history.

Under the TEE assumptions, attestation establishes that the Mint executed the approved authorization procedure.

6 The Resolver

The Zcash chain records Name Notes in canonical order, but it does not determine which transitions belong in the registry. A Resolver verifies each Name Note and applies accepted transitions to derive registry state.

Anyone can independently run a Resolver. Resolvers reading the same canonical Zcash chain under the current protocol rules must derive the same registry state.

6.1 Resolver responsibilities

A Resolver must:

1. scan the canonical Zcash chain using the Mint's published full viewing key to identify and decrypt Ironwood outputs, obtaining their memos and note components;
2. parse the decrypted outputs to identify Name Note candidates;
3. verify each candidate's note commitment using the ZNS-specific derivation of rcm and ψ defined in Section 3;
4. verify Mint authorization and attestation as specified in Section 4;
5. apply accepted transitions in canonical order to derive registry state; and
6. provide clients with the current and historical binding state of each name, including the `expires_at` value committed by the applicable Name Note.

If a chain reorganization changes the canonical Zcash history, the Resolver MUST roll back any state derived from removed blocks and replay the replacement chain from the common ancestor.

6.2 Candidate verification

Before state derivation, the Resolver verifies two properties of each Name Note candidate:

1. Name Note verification. The Resolver parses the encoded transition and reconstructs the note commitment as specified in Section 3. The candidate passes Name Note verification only if the reconstructed note commitment matches the `cmx` recorded on-chain.

2. Mint authorization. The Resolver uses the Mint's published viewing key to identify Name Notes created by the Mint and verifies the Mint's attestation as specified in Section 4.

User authorization is not independently reconstructed by the Resolver. As described in Section 5, the OTP exchange does not produce public evidence of an individual authorization event. Under the TEE assumptions, the Resolver relies on Mint attestation to establish that the approved authorization procedure was executed.

6.3 State derivation

The Resolver processes verified Name Note candidates in ascending canonical block order. Within a block, candidates are ordinarily processed according to transaction and action order, subject to the same-block release precedence rule below.

For each name, the `prev_rcm` field links a transition to the preceding Name Note within the current registration. A `claim`, including one following a `release`, uses the zero predecessor defined in Section 3. An `update` or `release` is accepted only if its `prev_rcm` matches the `rcm` of the current accepted Name Note.

If the same block contains both one or more `update` candidates and one or more `release` candidates for the same name that reference the `rcm` of the same current accepted Name Note, the Resolver considers the `release` candidates before the competing `update` candidates regardless of their relative transaction order within that block.

If more than one competing `release` references that predecessor, the releases are considered according to transaction and action order. Once one release is accepted, the registration ends and the remaining competing transitions

referencing the former predecessor do not affect registry state.

An `update` therefore affects the registration only if it is accepted in a block preceding the block containing a competing `release`. An update accepted in the same block as that release does not affect registry state.

Each accepted transition updates the registry state for that name. A transition whose predecessor does not match the current accepted state does not affect the registry.

7 Failure Domains

ZNS is an overlay network. It achieves consensus without requiring Zcash nodes to evaluate name registry rules. To achieve this, the architecture accepts specific failure domains as calculated trade-offs.

7.1 The Execution Domain: TEE vs. ZKP

A zero-knowledge proof of a state transition is mathematically pure, but enforcing a globally unique namespace requires proving a negative: that a requested name is not currently registered by anyone else.

If a decentralized protocol relies on client-side ZK-SNARKs, the user's device must prove non-inclusion against the global state root. Doing this privately requires Private Information Retrieval (PIR) inside the circuit, which is computationally prohibitive for a mobile wallet. Furthermore, pushing the registry rules into a client-side circuit requires wallet consensus: every major Zcash wallet would have to bundle and execute a heavy, protocol-specific proving circuit.

Conversely, off-chain ZK-Rollups that execute server-side proofs reintroduce the trusted sequencer and rely on infinitely complex, upgradable smart contracts.

ZNS trades cryptographic purity for immediate viability. By isolating the state machine inside a Trusted Execution Environment (TEE), the wallet's job is trivialized to sending a standard Orchard memo. The failure domain shifts entirely to the hardware vendor (e.g., an Intel or AMD side-channel compromise). If the TEE is breached, the attacker can forge Mint signatures and overwrite the registry.

7.2 The Availability Domain: On-Chain vs. Off-Chain

Storing the registry state off-chain and merely anchoring a Merkle root on-chain is standard practice for scalable overlay networks. ZNS rejects this model.

If ZNS stored state off-chain, the Mint would become a permanent hostage situation. If the Mint operator disappeared or suffered catastrophic hardware failure, the off-chain database would be lost. The on-chain Merkle root would be useless for reconstructing the namespace, rendering the protocol permanently dead.

ZNS forces every state transition into the strict 512-byte payload limit of an Orchard memo. The failure domain is Zcash consensus itself. If the Mint is vaporized, any developer can spin up a Resolver, scan the Zcash blockchain, and perfectly reconstruct the final registry state from the canonical history of Name Notes.

7.3 The Authorization Domain: OTP vs. Signatures

Zcash shielded addresses lack a standard message-signing protocol. Even if one existed, a raw cryptographic signature proves ownership of a key, but it does not prove temporal liveness, nor does it prevent replay attacks without a persistent on-chain nonce registry.

ZNS uses an in-band One-Time Passcode (OTP). The Mint encrypts 16 bytes of entropy and sends it to the Unified Address on record via a standard Zcash shielded transaction. Returning that exact payload to the Mint proves two things simultaneously: cryptographic read-access to the controller address, and temporal liveness bounded by the OTP expiration block.

The failure domain is key loss. If a user loses the viewing key to their Unified Address, they can never read the OTP. The name is permanently locked and cannot be updated or transferred until its registration interval lapses and the Mint returns it to the public pool.

8 Privacy Considerations

The public Name Note viewing key exposes registry contents to any scanner. The protocol does not provide name-binding confidentiality. Its payment-privacy claim is limited to the absence of a transparent transaction graph for ordinary shielded payments to the bound Unified Address. Network observers, the Mint, wallet providers, and authorization infrastructure can observe metadata outside that claim.

Out-of-scope privacy properties are recorded in Section **??**: registration privacy, resolution privacy, and freshness to a non-replaying client are not properties of this construction.

9 Deployment Parameters

The byte encodings and reducer do not determine which deployment an implementation follows. The initial active protocol manifest MUST fix the remaining deployment and policy values below.

Area	Required manifest value or referenced rule
Activation and genesis	Network identifier, activation height, Name Note viewing key, scan birthday, initial Mint account parameters, and initial recognized Mint note set.
Claim eligibility	Availability rule, price, payment asset, amount, payment recipient, required confirmations, and binding between payment and request.
Renewal	Renewal interval `renewal_interval` in blocks, refresh action, lapse rule, and renewal-fee policy.
Authorization transcript	Shielded receiver type, OTP relay memo grammar, OTP entropy source, OTP validity window ‘N‘ in blocks, OTP attempt limit, and one-time OTP consumption.
Attestation	Approved measurements, audited source revisions, immutable audit reports, verifier roots, minimum security versions, debug rejection, Mint-key binding, revocation, and key rotation.
TEE state	Sealed-state format, rollback detection, backup, restart, and compromise-recovery procedure.
Resolution	Response encoding, evidence encoding, confirmation policy, finality policy, and freshness limit.
ZNSIP release authority	Initial editor list, proposal repository, manifest publication location, and the decision process used to select a manifest for distribution.
Conformance corpus	Positive, negative, stale, conflicting, malformed, and reorganization vectors for every normative encoding and transition.

Each value is identified by the manifest content hash. A software release date or operator announcement is not a protocol activation rule.

10 ZNS Improvement Proposals

A source-code change cannot tell independent implementations when a new rule begins or which earlier bytes retain their old meaning. ZNS therefore

uses Zcash Name Service Improvement Proposals, abbreviated ZNSIPs, to specify protocol and process changes. The workflow follows the separation between publication, review, and deployment used by the ZIP process [6].

10.1 Proposal scope

One ZNSIP defines one change. A patch that affects only one implementation and does not affect interoperability does not require a ZNSIP. A change to any item below requires a Standards ZNSIP:

- canonical-name or wallet-resolution rules;
- memo, hash-transcript, field, or commitment encoding;
- candidate validity, Mint origin, lifecycle, ordering, or reducer behavior;
- authorization or attestation policy interpreted by another implementation;
- activation, manifest, or migration behavior; or
- a wallet or Resolver interface required for interoperability.

A Process ZNSIP changes the ZNSIP workflow or release process. An Informational ZNSIP records analysis without imposing a conformance requirement.

10.2 Required document fields

Every ZNSIP contains a number, title, owner list, status, category, creation date, license, discussion link, and dependency list. The editors assign numbers; an owner MUST NOT self-assign one. The repository filename is `znsip-NNNN.md`, where `NNNN` is the four-digit assigned number. The body contains Motivation, Specification, Rationale, Security Considerations, Privacy Considerations, Compatibility, Test Vectors, Reference Implementation, and Activation. A section that does not apply states why it does not apply.

A Standards ZNSIP that changes bytes or cryptographic computation MUST include positive and negative cross-language vectors. A Standards ZNSIP that changes trusted code, attestation, key custody, or authorization MUST identify the public security review required before activation. A Standards ZNSIP that changes state derivation MUST define migration, rollback, and behavior across the activation boundary.

10.3 Status workflow

The permitted statuses are `Draft`, `Review`, `Accepted`, `Active`, `Rejected`, `Withdrawn`, and `Superseded`.

`Draft`. The proposal may change without preserving interoperability.
`Review`. The owners assert that the specification and required vectors are complete.
`Accepted`. The editors have recorded the review decision and its unresolved objections. Acceptance does not activate the proposal.
`Active`. An active protocol manifest includes the proposal and its activation condition has been reached.
`Rejected`. The recorded decision declines the proposal.
`Withdrawn`. The owners have stopped pursuing the proposal.
`Superseded`. A later ZNSIP identifies the replacement proposal.

At least two editors maintain numbering, format, status records, and the versioned proposal repository. Editors MUST NOT use publication review to rewrite a proposal's technical decision. Every status change after `Draft` MUST identify the repository revision and a public decision record. The owners may move a proposal from `Draft` to `Review` after publishing the claimed-complete revision. The review period is at least 30 calendar days. After that period, at least two-thirds of the listed editors, and no fewer than two editors, must approve an `Accepted` or `Rejected` decision. An editor who owns the proposal does not vote on that decision and is excluded from the denominator.

10.4 Protocol manifests and activation

The encodings and reducer in this specification are portable across deployments. An implementation obtains its rule set from an *active protocol manifest* selected by the operator or software distribution. Manifest selection is a software-distribution trust decision; the Zcash chain does not select a ZNS manifest.

The manifest identifies all of these values:

1. a manifest identifier and predecessor manifest identifier;
2. the Zcash network and activation height;
3. every included Standards ZNSIP and its immutable content hash;
4. the Name Note viewing key, initial Mint-controlled note set, and scan

birthday;
5. accepted Mint identifiers and attestation policy artifacts;
6. the domain tags and protocol-version values used by the included rules;
7. the claim-eligibility, authorization, sealed-state, and resolution parameter values fixed by this deployment; and
8. hashes of the conformance vectors.

A Resolver MUST expose its manifest identifier and activation height with every resolution response. Two results computed under different manifests are not claims about the same protocol state. The manifest identifier is the lowercase hexadecimal digest of the exact published manifest bytes under unkeyed BLAKE2b with a 32-byte output and no personalization. Each ZNSIP content hash uses the same construction over the exact repository file bytes. A software release date or operator announcement is not a protocol activation rule.

An accepted Standards ZNSIP has no normative effect unless an active manifest includes its exact content hash. At activation height, the Resolver applies the new rules to occurrences at that height and later heights. It MUST apply the predecessor manifest to every earlier occurrence unless the ZNSIP defines an explicit migration from the already-derived state. A ZNSIP MUST NOT retroactively reinterpret an earlier occurrence by silence.

A memo or commitment change MUST use a new domain tag or an otherwise unambiguous version discriminator. A parser MUST NOT guess a version from malformed input. If a reorganization removes blocks at or above the activation height, the Resolver rolls back state under the rules that applied to each disconnected height and then replays the replacement chain under the same height boundary.

The initial publication of this process is ZNSIP-0000. The initial normative protocol specification is ZNSIP-0001.

11 Glossary

Terms defined in the body are gathered here for reference. Each is normative where it is defined; this section restates nothing.

Term	Definition or defining section
ZNS (Zcash Name Service)	The protocol this document specifies; see Section 1.
Zcash name	The wallet-facing string `n.zcash` for a canonical name `n`; see Section **??**.
Canonical name	The byte string from which a ZNS name is built; see Section **??**.
Name Note	An Orchard note whose memo carries one canonical ZNS tuple and whose note commitment is derived from the same tuple; see Section 3.
Action	One of `claim`, `update`, or `release`; see Section 3.
Predecessor (`prev_rcm`)	The 32-byte rcm_σ of the accepted prior Name Note for the same name, encoded as 64 lowercase hex characters; see Section 3.
Accepted tip	The most recently accepted Name Note for a given name at a canonical chain position; see Section **??**.
Binding (current binding)	The Unified Address in the accepted live tip, or none while released; see Section **??**.
Wallet	Requests a name action and completes the authorization exchange; see Section 2.
Mint	Performs the policy checks in Section **??** before creating a Name Note; runs inside a TEE; see Section 2.
Resolver	Performs the scan in Section **??** and applies the state reducer; see Section 2.
Attested Mint policy	Remote attestation binding enclave measurement, Mint authority, protocol version, network, and protected-state scheme; see Section 2.
Active protocol manifest	The deployment-selected bundle of protocol version, Zcash network, activation height, viewing key, Mint note set, and field values; see Section 10.
ZNSIP	A ZNS Improvement Proposal, processed per Section 10.
Name Note viewing key	The published Orchard full viewing key of the Name Note account; see Section 1.
One-time passcode (OTP)	A 16-byte random value issued by the Mint to a controller receiver and returned by the wallet to authorize an `update` or `release`; see Section 5.
OTP relay memo	The Orchard shielded memo `ZNS:otp:<name>:<verb>:<ua>:<otp>` carrying an OTP to the controller receiver; see Section 5.

External terms inherited from the Zcash protocol specification [1] or its ZIPs are not redefined here: *Orchard action*, *Orchard memo*, *note commitment* (`cmx`), *Unified Address*, and *incoming viewing key* retain the meaning fixed by their normative sources [3, 4, 1].

References

[1] Electric Coin Company and Zcash Foundation. *Zcash Protocol Specification*. https://zips.z.cash/protocol/protocol.pdf.

[2] Zcash Improvement Proposal 212. *Allow Recipient to Derive Ephemeral Secret from Note Plaintext*. https://zips.z.cash/zip-0212.

[3] Zcash Improvement Proposal 224. *Orchard Shielded Protocol*. https://zips.z.cash/zip-0224.

[4] Zcash Improvement Proposal 316. *Unified Addresses and Unified Viewing Keys*. https://zips.z.cash/zip-0316.

[5] S. Bradner and J. Leiba. *Key words for use in RFCs to Indicate Requirement Levels*. BCP 14, RFC 2119 and RFC 8174. https://www.rfc-editor.org/info/bcp14.

[6] Zcash Improvement Proposal 0. *ZIP Process*. https://zips.z.cash/zip-0000.

[7] zcashme. *zns-mint reference implementation*. https://github.com/zcashme/zns-mint.